

SMARTSUPER: SUPERMARKET INVENTORY CONTROL USING SORTING, MERGING, AND FILE PROCESSING

PRESENTED BY,

VAHINI.M (192412116)

JANANI.E (192511325)

SWATHI.S (192512333)

GUIDED BY,

DR.V.KARPAGAM

INTRODUCTION

1. Supermarkets handle large volumes of products daily
2. Efficient inventory control helps avoid overstock and stock-outs
3. This project automates product management using Sorting, Merging, and File Processing in C
4. Covers two modules:
 - Inventory Data Management
 - Supplier Data Integration



INVENTORY CONTROL

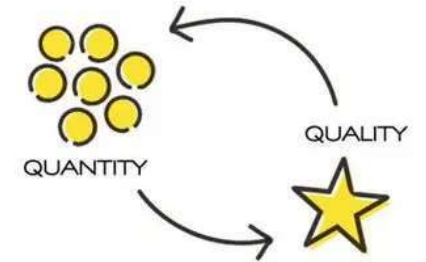
PROBLEM STATEMENT

1. Manual inventory updates lead to errors and missing data
2. Unsorted product lists slow down searching & reporting
3. Supplier stock updates are hard to merge manually
4. Need for a simple automated C-based system to handle inventory efficiently



OBJECTIVES

1. Store and manage product data using file handling.
2. Sort inventory based on product name, price, or quantity
3. Merge supplier file with supermarket inventory
4. Automatically update quantities and add new items
5. Reduce human error and save time



LITERATURE REVIEW

1. Khakbaz A. et al. (2024) A Novel Cross-Docking EOQ-Based Model to Optimize a Multi-Item Multi-Supplier Multi-Retailer Inventory Management System.
2. Li A.A., Peng T. (2024) Fixing Inventory Inaccuracies at Scale.
3. Ping H. et al. (2024) Optimization of Vegetable Restocking and Pricing Strategies for Innovating Supermarket Operations Utilizing ARIMA, LSTM, and FP-Growth Algorithm.
4. Senyuva I.E., Drent M., Atan Z. (2024) Perishable Inventory Management Under Freshness-Dependent Demand and Backroom Effect.
5. Bai Y. et al. (2025) Coordinated Inventory Stocking and Assortment Customization.

INPUT

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width,initial-scale=1" />
<title>Supermarket Inventory – Dashboard (Dark Neon)</title>
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&display=swap" rel="stylesheet">

<style>
:root{
  --bg:#0b0f1a;
  --card:#0f1724;
  --muted:#9aa7c7;
  --accent:#2ec7ff;
  --accent-2:#7c4dff;
  --glass: rgba(255,255,255,0.03);
  --success:#28c76f;
  --danger:#ff6b6b;
  --radius:12px;
}
*{box-sizing:border-box}
body{
  margin:0;
  font-family:"Inter",system-ui,Arial;
  background: radial-gradient(800px 400px at 10% 10%, rgba(46,199,255,0.06), transparent),
    radial-gradient(800px 400px at 90% 90%, rgba(124,77,255,0.05), transparent),
    var(--bg);
  color:#dbe7ff;
  -webkit-font-smoothing:antialiased;
  -moz-osx-font-smoothing:grayscale;
  min-height:100vh;
}

/* layout */
.app {
  display:flex;
  gap:24px;
  padding:28px;
```

```
padding:28px;
min-height:calc(100vh - 56px);
}

/* sidebar */
.sidebar {
  width:260px;
  background:linear-gradient(180deg, rgba(255,255,255,0.02), rgba(255,255,255,0.01));
  border-radius:var(--radius);
  padding:10px;
  box-shadow: 0 8px 30px rgba(2,8,23,0.6);
  border:1px solid rgba(255,255,255,0.03);
  position:relative;
}

.brand {
  display:flex;
  gap:12px;
  align-items:center;
  margin-bottom:18px;
}

.logo{
  width:44px;height:44px;border-radius:10px;
  background:linear-gradient(135deg,var(--accent),var(--accent-2));
  display:flex;align-items:center;justify-content:center;
  font-weight:700;color:#031029;box-shadow:0 6px 18px rgba(124,77,255,0.18);
}

.brand h3{margin:0;font-size:16px}
.brand p{margin:0;color:var(--muted);font-size:12px}

.nav {
  margin-top:8px;
  display:flex;flex-direction:column;gap:8px;
}

.nav button {
  display:flex;gap:12px;align-items:center;padding:10px 12px;border-radius:10px;border:none;
  background:transparent;color:var(--muted);cursor:pointer;text-align:left;font-weight:600;
}

.nav button.active, .nav button:hover {
  background: linear-gradient(90deg, rgba(46,199,255,0.06), rgba(124,77,255,0.03));
  color:var(--accent);
  box-shadow: inset 0 -1px 0 rgba(255,255,255,0.02);
}

.nav button svg{opacity:0.9}

.sidebar .small {
  margin-top:18px;
  font-size:12px;color:var(--muted);
}

/* main content*/
.main {
  flex:1;
  display:flex;flex-direction:column;gap:18px;
}
```

OUTPUT

Products

Sort by ID
Sort by Name
Sort by Qty

Search matches: 2

ID	Name	Category	Qty	Price (₹)	Supplier	Actions
-2	Apple	Grocery	2	₹23.00	Hasya	Edit Delete
-4	Orange	Grocery	5	₹28.00	Hasya	Edit Delete

Search product name / id ...

Merge Preview

ID	Name	Qty	Price	Supplier
2	Apple	2	₹23.00	Sushmitha

Supplier — Add Items

Supplier Product ID (numeric)	Product Name	Quantity
Price (₹)	Supplier Name	Category — select

Add Supplier Item
Clear

ID	Name	Qty	Price	Supplier	Actions
-2	Apple	3	₹23.00	Sushmitha	Remove

Merge Supplier to Inventory
Clear Suppliers

Search product name / id ...

Welcome H

File Export / Import (Local)

Download CSV
Save to Local (simulate file)
Load from Local
Choose File
No file chosen
Import CSV

Tip: CSV can be opened in Excel. Save/Load uses browser localStorage so your data remains on this browser.

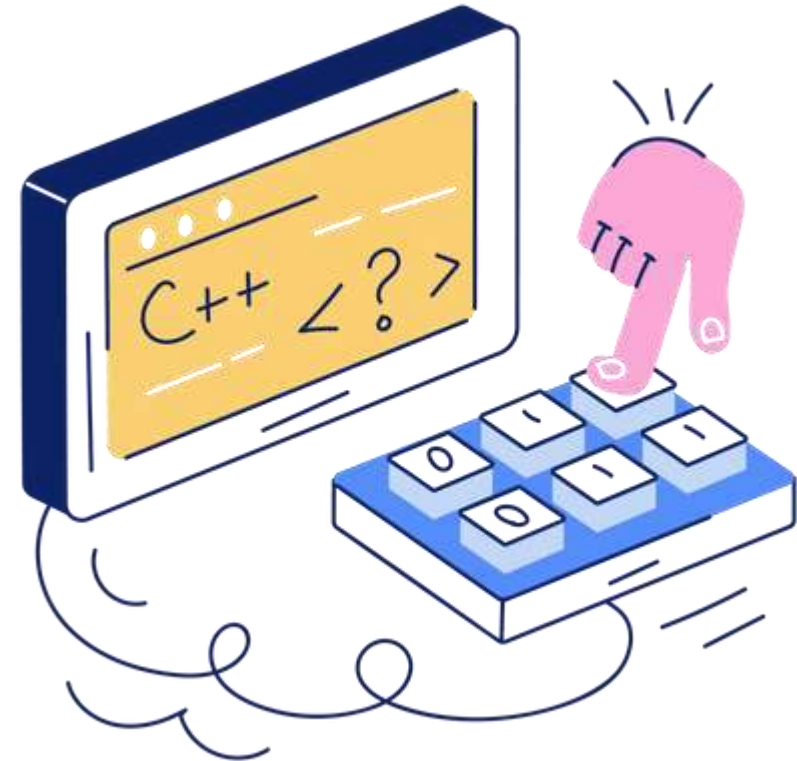
MODULE 1 : INVENTORY DATA MANAGEMENT

OVERVIEW

1. Maintains supermarket's main inventory file

- Performs:
- Add product
- Update quantity
- Delete product
- Search product
- Display inventory
- Sort inventory list

2. Uses C file functions for storage and retrieval



SORTING TECHNIQUES USED

- Sorting keeps inventory neat and easy to search
- Implemented in C using:
 - Bubble Sort
 - Selection Sort (optional)
- Sorting based on:
 - Product Name (A–Z)
 - Quantity
 - Price
- Helps generate organized inventory reports



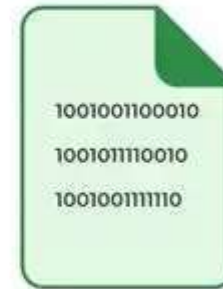
FILE PROCESSING

1. Uses FILE*, fopen(), fprintf(), fscanf(), fclose().
2. Stores each product as a record containing:
 - ID
 - Name
 - Quantity
 - Price
 - Advantages:
 - Permanent storage
 - Faster access
 - Error-free updates
 - Easy retrieval for sorting and merging

Types of Files in C



file.txt

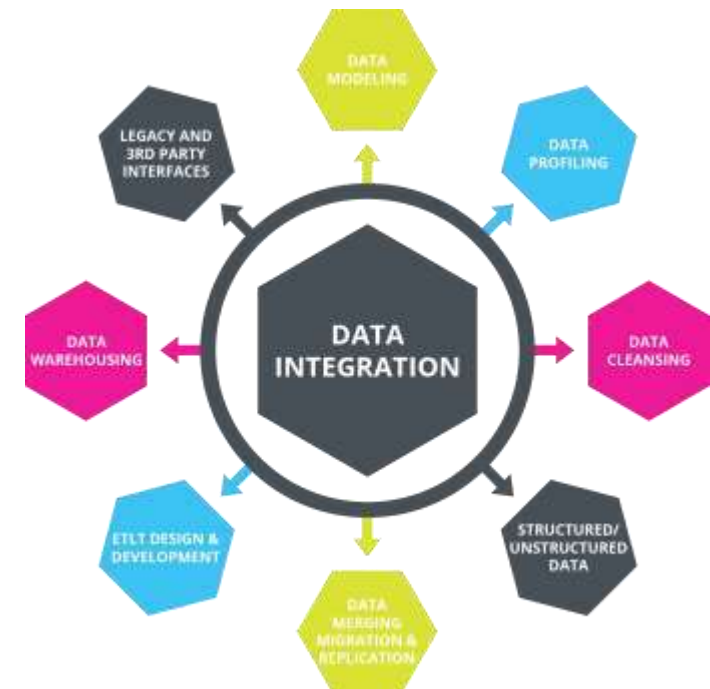


file.bin

MODULE 2 : SUPPLIER DATA INTEGRATION

OVERVIEW

- Reads supplier stock list from a separate file
- Compares supplier's product ID with inventory
- If ID matches → quantity is updated
- If new product → added to inventory
- Ensures smooth restocking and correct stock counts



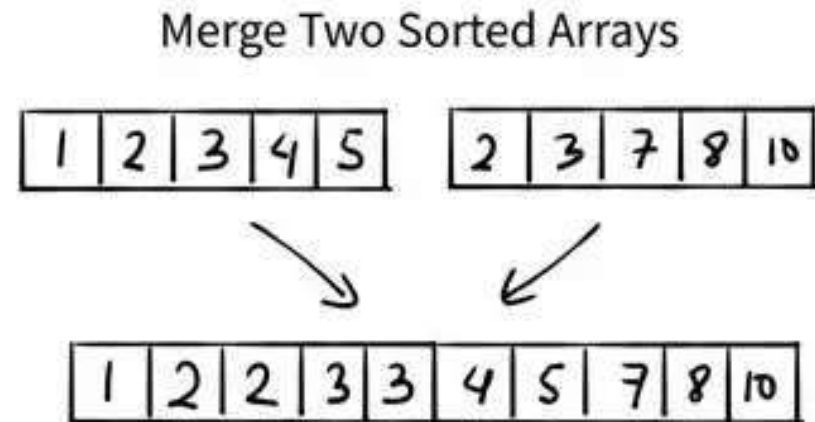
MERGING TECHNIQUES

- Follows the logic of merging two sorted files

- Steps:

1. Open inventory file
2. Open supplier file
3. Compare product IDs
4. Update or add item
5. Write final data to updated file

- Makes merging faster and avoids duplicate entries



MODULE 3 : INVENTORY REPORTING & STOCK ANALYSIS

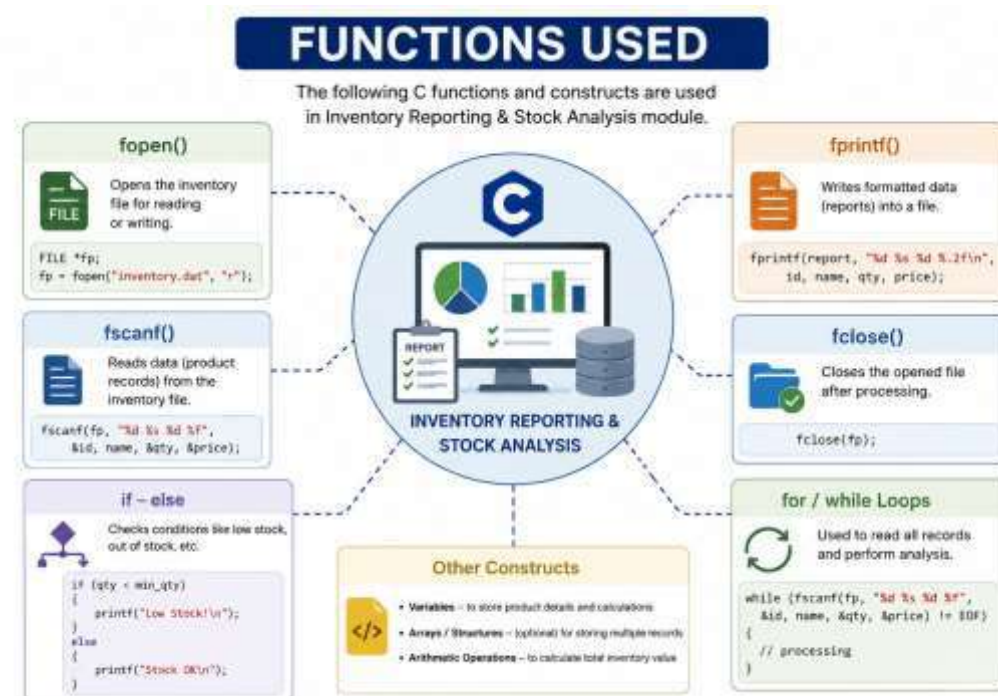
OVERVIEW

- Generates inventory summary reports
- Displays low-stock products.
- Calculates total inventory value.
- Shows out-of-stock items.
- Helps management make restocking decisions.



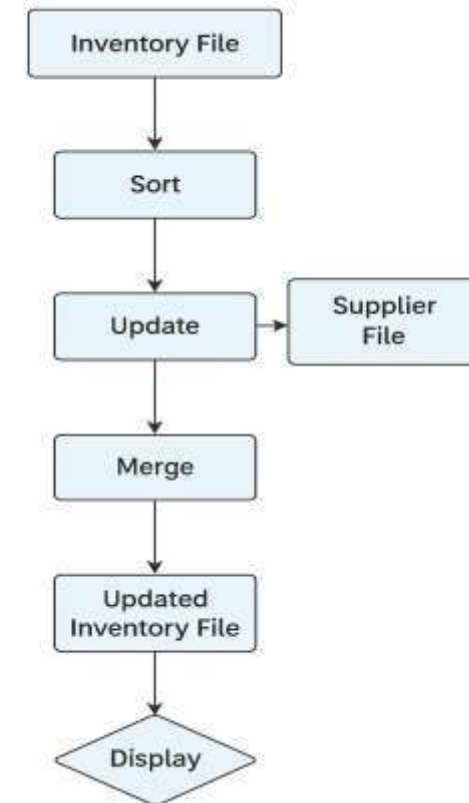
FUNCTIONS USED

- `fopen()`
- `fscanf()`
- `fprintf()`
- `fclose()`
- `if-else`
- `for / while loop`



CONCLUSION

1. The system automates supermarket inventory control
2. Sorting + merging improves accuracy and efficiency
3. C file handling ensures reliable storage and updates
4. The project can be extended for billing, GST, and database systems



THANK YOU